32XCD Boot Engine - Lessons Learned Addendum

Written after an extended real-hardware debugging effort on a Sega CD + 32X + MicroNut99 4MB RAM cart project. Intended as a companion to, and cross-checked against, two reference documents the project's developer (MicroNut99) supplied: `KOBO-CD32X_BOOT_PROCESS.pdf` ("Kobo Deluxe Sega CD32X - How It Boots and Runs") and `KOBO-Sega_32XCD_Boot_Sequence_and_Sub-CPU_Command_Ledger.pdf`. Both describe a more mature, working sibling project in the same lineage, built on the same four- processor architecture. Where this addendum states something as fact rather than hypothesis, it has been checked against one of those two documents, the official Sega 32X Hardware Manual, or real on-console behavior - not assumed.

1. The architecture, once it clicks, is simple - the bugs live in the handoffs

Four processors, each with a narrow, exclusive capability, relaying requests in a chain:

```
SH2 (the game/logic)  <-->  Sub-CPU (the coordinator)  <-->  Genesis 68000
(command slave)
     via 32X COMM regs              via Sega CD COMM regs
```

- The **SH2** cannot reach the CD or Sega CD hardware at all.
- The **Sub-CPU** cannot reach the 32X registers, the Genesis VDP, or the cart.
- The **Genesis 68000** can reach both the 32X and the cart, but has no CD access.

Every request that needs something outside the asking processor's own reach has to cross at least one of these boundaries, and every boundary is a place where timing assumptions can quietly break in ways that only show up on real silicon. This single fact explains most of a long night's worth of confusing symptoms: a freeze that looked like it was "in the picture-loading code" was actually in the round trip the picture-loading code depended on.

2. Comm registers: direction matters more than it looks like it should

The official 32X Hardware Manual (p.69 in the dev's copy) states plainly that a comm register written by one CPU while the other reads it is **undefined** - and does not specify which way a collision fails. This is not a theoretical concern. The Kobo document's own hardware-rules section states it as a lesson learned on real hardware, independently: *"do not hammer a register while the other CPU writes it. Use one writer per register, poll with pauses, and avoid 32-bit writes that also hit another CPU's half (COMM12/14)."*

This project made exactly this mistake once, in a hand-built isolated-interrupt test: one comm register (COMM10) was written from both the SH2 side and the 68K side, each polling it while the other might be mid-write. The practical effect was a clean-looking but silent failure - the test reported "never answered" every time, with no corruption visible, no crash, just a result that was impossible to distinguish from a genuinely broken mechanism. The fix was to give each direction its own register and never share one. **Treat every comm**

register as having exactly one writer. If a reply is needed, it goes on a different register than the request.

A related, sharper trap: COMM12 and COMM14 are not fully independent - on the SH2 side they are also addressable as one combined 32-bit register (`COMM12_32`, spanning both). A 32-bit write from one side can clobber a 16-bit value the other side just wrote to only one half. If both are in use for anything at the same time, know which access width each side is actually using.

3. RV and the cart: byte vs. word is not pedantry, it changes which bits you're touching

The RV bit (the mechanism that lets the 68K safely access the cart by halting the SH2 the instant it touches cart ROM) lives at bit 0 of a *word* register whose base address is `$A15106`. The byte at the odd address `$A15107` is that same word's low byte, so a **byte**-sized `bset #0,$A15107` / `bclr #0,$A15107` correctly hits RV - and this is exactly what the manual's own BIOS listing does. Treating `$A15107` as if it were itself the base of a *word* register would not just be redundant, it would be wrong - the true word register starts one byte earlier. This project's real-hardware collaborator (Chilly Willy, the original 32X/Sega CD toolchain author) flagged this distinction specifically and it checked out exactly against the manual's own example code. Get the access width right for whichever bit you're actually touching, not just the address.

The companion interrupt control register (`$A15102`, used to assert a Command Interrupt on either SH2) explicitly supports both byte and word access per the manual - a genuinely different case from RV, not a contradiction. Don't assume both registers follow the same rule; check each one.

4. The cart has two independent addressing schemes for the same physical chip

This was the single most valuable, concrete fact recovered from the official manual this session, and it is worth stating as plainly as possible:

- **From the SH2's own side**, the cart is a fixed, simple, linear address: `0x22000000` (cache-through) or `0x02000000` (cached) **plus however many bytes into the cart you want** - confirmed in Figure 3.2 of the manual ("SH2 Memory Map"), and independently confirmed in the Kobo document: *"the SH2 reads it at `0x22000000 + offset`."* This address never changes, regardless of what the Main CPU is doing.
- **From the Genesis 68000's side**, the window the cart appears in depends entirely on context: `0x880000-0x9FFFFF` (with bank switching via `$A15104`) for a plain cart-boot scenario, but a *different* window, `0x400000-0x7FFFFF`, specifically when booting from a Sega CD with the 32X active (confirmed directly by Chilly Willy, and independently corroborated in a 2011 SpritesMind.Net post by the same author describing the cartridge as mapping to `0x400000` "because the cartridge doesn't assert the 'cart in' line" in that mode). The Kobo project uses yet a third 68K-side window (`0x900000` via the bank register) for its own cart layout - all three are valid, context-dependent views of the same chip, not competing claims.

Do not combine these formulas. The SH2's address is never "the 68K's window address, plus the SH2 offset" - it is always just `0x22000000 + (byte offset into the cart itself)`, full stop. This project nearly shipped a test that accidentally double-counted the two addressing schemes before the mistake was caught by asking precisely what "the cart's address" meant on each side separately rather than assuming one formula covered both.

5. Startup handshakes are exactly where "it worked a dozen times then didn't" bugs hide

The Kobo document records, as settled history, that an earlier version of this exact architecture used a wait on `COMM4 == 0x1234` during SH2 startup - and that this wait raced against `crt0.s`'s own clearing of COMM4, and was removed in favor of a dedicated `0x0AAA/0x0BBB` exchange on a different register. This is worth internalizing as a general pattern, not just a specific fix: **a startup gate that happens to work during casual testing can still have a real race in it** - the failure mode of a race is, by definition, that it doesn't fail every time. If a future version of this project (or any 32XCD project) exhibits an SH2 that "sometimes doesn't wake up," checking every startup wait for exactly this shape of bug - two independent pieces of code touching the same flag with no ordering guarantee between them - is a better first move than assuming the problem is in whatever code happens to run right after the gate.

Related, from the same document's hardware-rules section: a long `.bss` clear in `crt0.s` once raced the start handshake outright. Startup-time races are not rare edge cases in this kind of four-processor system; they are close to the default failure mode, and worth testing for specifically rather than hoped past.

6. Bound every wait, but size the bound by what the wait actually costs

Every long-lived wait loop in this codebase that could plausibly never resolve - waiting for an SH2 to park, waiting for a cart write to complete, waiting for a startup handshake - needs a bound, or a single wrong assumption anywhere in a four-processor relay chain turns into an indefinite hang with zero diagnostic information. That part is close to non-negotiable for this kind of project.

But the bound has to be sized for what one pass of the loop actually costs, not copied from a different loop that looks similar. A loop that does one raw hardware-register read per pass and a loop that does a full cross-CPU round trip per pass (waiting on the Sub-CPU, which is itself waiting on the Genesis 68000) can differ in real cost by two or three orders of magnitude for the "same" iteration count. This project sized a cross-CPU-round-trip wait using the iteration count from an unrelated tight register-polling loop, and the result was a wait that looked identical in the source but took on the order of five real minutes instead of the intended few seconds. Where a real-time tick counter is already available (as it is here, incremented once per vblank), prefer bounding by elapsed real time over any raw iteration count - it fails in a way that's immediately legible, rather than in a way that requires re-deriving the actual hardware cost after the fact.

7. Diagnose in layers, on-screen, in the actual failure's path - not around it

A recurring, effective pattern across this project's hardest bugs: when a multi-step handshake fails as a single pass/fail result, it tells you almost nothing about *where* it failed. Splitting a composite test into its real sub-steps - and reporting each one's outcome on screen, not just the final verdict - turns "it didn't work" into "it got this far and no further," which is the difference between a mystery and a lead. This project used exactly this technique twice to real effect: separating "did the Command Interrupt parking handshake succeed" from "did the RV-bit cart access succeed" (two genuinely different mechanisms bundled into one earlier test), and separating "did the SH2 reach this code at all" from "did it reach the interrupt-enable instruction specifically" via a cheap checkpoint value written immediately before the step actually in question. Neither required new hardware insight - just refusing to let one pass/fail number stand in for several independent questions.

8. Emulators validate some things and actively cannot validate others - know which is which

Fusion successfully running a wide range of real CD32X titles for years is genuine, strong evidence that its 32X activation, frame-buffer handoff, and SH2-start sequence are solid - that part of the boot process is identical for every CD32X title, cart or no cart, and has been thoroughly exercised by real, shipped software. It is **not** evidence about narrow, cart-specific mechanisms like RV-bit timing or Command-Interrupt-based SH2 parking, because - as far as could be determined - no released CD32X title combines a banked SRAM cartridge with the CD32X boot path the way a RAM-cart project does. A newer, more accuracy-focused emulator (jgenesis) only added correct RV-bit emulation in a recent release, per its own changelog - concrete evidence this specific mechanism is genuinely hard to emulate correctly even when the surrounding boot process is right. A clean result on this narrow category of hardware interaction, in any emulator, should be treated as inconclusive rather than confirming; only real hardware actually answers the question.

References

- Sega 32X Hardware Manual (official), Figure 3.2 "SH2 Memory Map," and the Interrupt Control Register / RV-bit (`$A15106`/`$A15107`) and Command Interrupt (`$A15102`) sections.
- `KOBO-CD32X_BOOT_PROCESS.pdf` - "Kobo Deluxe Sega CD32X: How It Boots and Runs," sections 3 (boot, step by step), 6 ("Hardware rules learned on the console"), and 7 ("Leftovers in the code").
- `KOBO-Sega_32XCD_Boot_Sequence_and_Sub-CPU_Command_Ledger.pdf` - the full COMM-register and command-number reference table.
- Chilly Willy, direct correspondence (2026) and a corroborating 2011 SpritesMind.Net post on the same architecture.